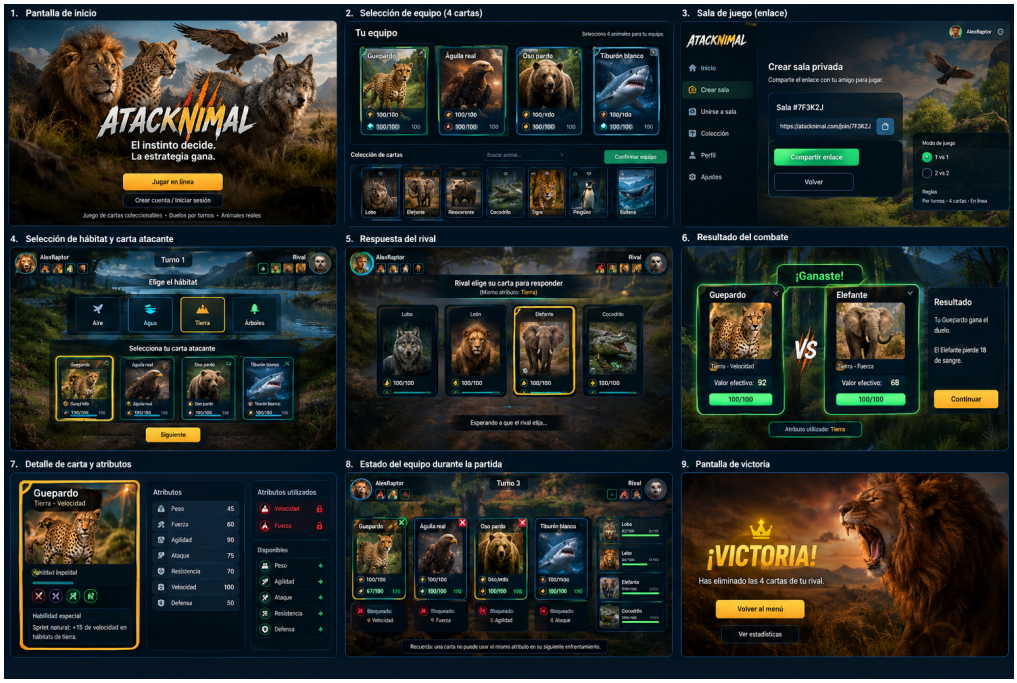
EL INSTINTO DECIDE. LA ESTRATEGIA GANA.



DOCUMENTO MAESTRO · VERSIÓN 4.0

Diseño de juego · Reglas · Balance · Animales · Hábitats · UX/UI · Mockups · IA · Multijugador · QR · Arquitectura · API de imágenes reales · QA · Roadmap

Documento completamente enfocado en los tres modos iniciales: 1 vs IA · 1 vs 1 · 2 vs 2. Sin economía, compras ni monetización.

1. Resumen ejecutivo

ATACKNIMAL es un videojuego 2D táctico de enfrentamientos entre animales reales representados mediante cartas. La partida no consiste en escoger simplemente el animal con la cifra más alta: el jugador debe interpretar el hábitat, seleccionar una carta y declarar el atributo que quiere utilizar. El rival conoce el hábitat + atributo, pero no conoce la carta atacante antes de escoger su propia carta defensora.

La primera versión debe ser pequeña, sólida y comprobable. La prioridad es que las reglas sean divertidas y estén perfectamente balanceadas antes de añadir sistemas secundarios.

Alcance: 1 vs IA, 1 vs 1 y 2 vs 2; equipos de cuatro animales; salas privadas mediante código/enlace/QR; imágenes reales obtenidas mediante APIs; combate por turnos; estados, daño y eliminación.

Decisiones tecnológicas

Capa	Tecnología	Responsabilidad
Aplicación	React + TypeScript	Navegación, menús, selección de equipo, sala, HUD, resultados y datos.
Combate visual	Phaser	Escenarios 2D, animaciones, transiciones, partículas y revelación del duelo.
Reglas	Motor independiente	Turnos, validaciones, cálculo, daño, restricciones y victoria.
Servidor	Node.js + API	Salas, partidas online, validación autoritativa y sincronización.
Datos	PostgreSQL	Animales, atributos, salas, partidas, turnos y resultados.

2. Pilares de diseño

• Decisión: cada turno debe obligar a elegir con intención.

• Información parcial: el defensor no conoce la carta atacante al tomar su decisión.

• Lectura de animales: cada especie tiene una identidad estadística reconocible.

• Hábitat relevante: el escenario modifica el rendimiento sin decidir automáticamente el ganador.

• Transparencia: después del duelo, el jugador entiende qué atributo se comparó y por qué ganó una carta.

• Espectáculo controlado: Phaser anima el resultado, pero nunca inventa la resolución.

• Balance medible: toda carta y atributo debe poder probarse mediante simulaciones.

Lo que NO forma parte de esta versión

• Economía

• Compras

• Tienda

• Sobres

• Monedas

• Gemas

• Marketplace

• Intercambio de cartas

• Monetización

Las cartas son unidades jugables. No son un producto económico en la primera versión.

3. Bucle principal de juego

El ciclo central debe ser corto y repetible: preparar equipo → iniciar turno → elegir hábitat → elegir carta → elegir atributo → revelar al rival solo hábitat + atributo → defensor elige carta → resolver → animar → aplicar daño/estado → actualizar equipo → siguiente turno.

Momento	Atacante	Defensor	Información visible
Inicio	Espera	Espera	Estado de partida
Declaración	Hábitat + carta + atributo	Espera	Solo al sistema
Ventana de defensa	Espera	Escoge carta	Hábitat + atributo
Revelación	Ve ambas cartas	Ve ambas cartas	Cartas + valores efectivos
Resolución	Recibe resultado	Recibe resultado	Ganador, daño, estado

4. Regla de turno — especificación exacta

A. Seleccionar hábitat — Aire, Agua, Tierra o Árboles.

B. Seleccionar carta atacante — Una carta disponible del equipo.

C. Seleccionar atributo — Uno de los ocho atributos definidos.

D. Cerrar elección — La combinación queda registrada y no puede cambiarse.

E. Notificar al defensor — El servidor envía únicamente hábitat + atributo.

F. Seleccionar defensa — El defensor elige una carta disponible.

G. Resolver — El motor calcula valores efectivos y compara.

H. Aplicar resultado — Daño, bloqueo, eliminación u otro estado.

I. Animar — Phaser presenta visualmente la resolución.

J. Continuar — Se cambia el turno según el orden de partida.

Regla de información crítica: el defensor NO puede ver el animal atacante antes de seleccionar su carta. Solo conoce el hábitat y el atributo. Esto es una mecánica central, no un detalle de interfaz.

Regla adicional derivada del storyboard

El storyboard muestra cartas que pueden quedar bloqueadas en un atributo después de un enfrentamiento. Por tanto, el motor debe contemplar el estado bloqueado por atributo y comunicarlo visualmente. Ejemplo conceptual: “Bloqueado: Velocidad”.

5. Atributos oficiales

Estos son los ocho atributos oficiales que deben utilizarse en el diseño de las cartas, el balance, la IA y el motor de combate.

Atributo	Representa	Perfil típico
Peso	Masa relativa	Grandes mamíferos/reptiles
Fuerza	Potencia física	Elefante, gorila, grandes depredadores
Agilidad	Maniobra y cambios de dirección	Felinos, zorros, primates
Ataque	Capacidad ofensiva	Depredadores y especies con armas naturales
Resistencia	Aguante	Animales capaces de mantener esfuerzo
Velocidad	Rapidez contextual	Guepardo, halcón, especies acuáticas rápidas
Defensa	Protección/evitación	Tortugas, armadillos, especies defensivas
Percepción	Detección del entorno	Águilas, zorros, especies con sentidos destacados

Escala recomendada

Para la primera iteración, todos los atributos se expresan de 1 a 100. El valor no pretende ser una medición científica exacta; es una escala de diseño para representar diferencias relativas y permitir balance.

Regla de identidad

Un animal no debe tener ocho estadísticas altas. Debe tener fortalezas, atributos secundarios y debilidades. La estrategia nace de decidir cuándo explotar una fortaleza y cuándo evitar un enfrentamiento desfavorable.

6. Hábitats y adaptación

Hábitat	Contexto	Ejemplos	Atributos especialmente relevantes
Aire	Vuelo, altura, maniobra aérea	Águila real, especies voladoras	Velocidad, Agilidad, Percepción
Agua	Nado, profundidad, desplazamiento acuático	Tiburón blanco, Ballena, Pingüino, Cocodrilo	Velocidad contextual, Resistencia, Ataque
Tierra	Superficie terrestre	Guepardo, Oso pardo, Lobo, Elefante, Rinoceronte, Tigre	Peso, Fuerza, Velocidad, Ataque
Árboles	Copa, ramas, verticalidad	Adaptación arbórea o de trepa	Agilidad, Percepción, Defensa

El hábitat modifica el rendimiento mediante un modificador contextual. No debe convertirse en un multiplicador tan grande que invalide las estadísticas del animal.

Modelo de diseño recomendado

Cada animal puede tener afinidad fuerte, neutral o débil por cada hábitat. El motor aplica un modificador pequeño y controlado al atributo seleccionado. La fórmula exacta debe quedar parametrizada para balancearla sin reescribir el sistema.

7. Roster inicial completo

La siguiente es la lista completa de animales disponibles en el roster inicial definido por el storyboard. Son 11 especies. Los valores son una propuesta de balance inicial y deben validarse mediante simulación.

Animal	Hábitat principal	Peso	Fuerza	Agilidad	Ataque	Resistencia	Velocidad	Defensa	Percepción	Fortalezas
Guepardo	Tierra / Árboles (situacional)	30	55	98	82	65	100	50	80	Velocidad / Agilidad
Águila real	Aire	20	55	95	75	60	95	55	100	Percepción / Agilidad
Oso pardo	Tierra	92	92	55	88	95	55	90	80	Fuerza / Resistencia
Tiburón blanco	Agua	85	90	65	96	90	92	75	85	Ataque / Velocidad
Lobo	Tierra	55	72	82	82	85	88	65	92	Resistencia / Percepción
Elefante	Tierra	100	100	40	82	95	40	98	75	Peso / Fuerza
Rinoceronte	Tierra	95	96	45	90	90	45	95	65	Defensa / Fuerza
Cocodrilo	Agua / Tierra	78	88	45	94	88	70	90	72	Ataque / Defensa
Tigre	Tierra / Árboles (situacional)	68	88	90	96	75	92	72	88	Ataque / Agilidad
Pingüino	Agua / Tierra (limitado)	35	50	75	55	82	65	70	88	Resistencia / Percepción
Ballena	Agua	100	95	50	80	100	85	88	90	Resistencia / Peso

Importante: estos números son estadísticas de diseño, no datos zoológicos. El juego busca crear decisiones equilibradas, no representar una tabla científica.

8. Definición funcional de cada animal

Guepardo

Especialista de velocidad. Debe ganar cuando el contexto favorece rapidez, pero es vulnerable frente a animales muy resistentes o defensivos.

Perfil: Alta: Velocidad/Agilidad. Media: Ataque/Percepción. Baja: Defensa/Peso. · Hábitat: Tierra · Balance: No permitir que Velocidad sea una solución universal.

Águila real

Especialista aéreo y de percepción. Su ventaja debe venir de leer el entorno y explotar Aire.

Perfil: Alta: Percepción/Agilidad/Velocidad. Media: Ataque. Baja: Peso/Resistencia. · Hábitat: Aire · Balance: Debe ser fuerte en Aire, pero no dominar Tierra.

Oso pardo

Tanque físico. Destaca por fuerza, peso y resistencia.

Perfil: Alta: Fuerza/Resistencia/Defensa. Media: Ataque. Baja: Velocidad/Agilidad. · Hábitat: Tierra · Balance: Evitar que sus tres estadísticas defensivas lo hagan invencible.

Tiburón blanco

Depredador acuático de alto ataque y velocidad contextual.

Perfil: Alta: Ataque/Velocidad/Fuerza. Media: Resistencia/Percepción. Baja: Agilidad fuera del agua. · Hábitat: Agua · Balance: Su poder debe depender del contexto acuático.

Lobo

Animal versátil, con percepción y resistencia superiores al promedio.

Perfil: Alta: Resistencia/Percepción/Agilidad. Media: Ataque/Velocidad. · Hábitat: Tierra · Balance: Su versatilidad no debe superar a especialistas.

Elefante

Especialista de masa y potencia física.

Perfil: Alta: Peso/Fuerza/Defensa/Resistencia. Baja: Velocidad/Agilidad. · Hábitat: Tierra · Balance: Debe sufrir contra velocidad y maniobra.

8. Definición funcional de cada animal — continuación

Rinoceronte

Especialista de defensa y ataque frontal.

Perfil: Alta: Defensa/Fuerza/Ataque. Media: Resistencia/Peso. Baja: Velocidad/Agilidad. · Hábitat: Tierra · Balance: No convertir Defensa en una estadística absoluta.

Cocodrilo

Depredador de emboscada con ataque y defensa.

Perfil: Alta: Ataque/Defensa/Fuerza. Media: Resistencia. Baja: Agilidad. · Hábitat: Agua/Tierra · Balance: Debe depender del contexto y de su oportunidad de ataque.

Tigre

Especialista ofensivo móvil.

Perfil: Alta: Ataque/Agilidad/Velocidad. Media: Percepción. Baja: Defensa relativa. · Hábitat: Tierra · Balance: Debe tener alto riesgo si el rival fuerza atributos defensivos.

Pingüino

Especialista de resistencia/percepción en agua.

Perfil: Alta: Resistencia/Percepción. Media: Agilidad/Defensa. Baja: Ataque/Fuerza. · Hábitat: Agua · Balance: Necesita contexto para ser competitivo.

Ballena

Especialista de masa, resistencia y percepción acuática.

Perfil: Alta: Peso/Resistencia/Percepción. Media: Defensa/Velocidad. Baja: Agilidad. · Hábitat: Agua · Balance: Debe ser poderosa sin ser una carta de respuesta universal.

9. Matriz de balance inicial

Para que el roster sea saludable, cada animal debe tener al menos un camino claro para ganar y varios caminos para perder. La tabla siguiente establece una primera lectura de roles.

Animal	Rol	Fortaleza primaria	Fortaleza secundaria	Debilidad principal	Contra ideal
Guepardo	Especialista veloz	Velocidad	Agilidad	Defensa	Oso/Elefante en Resistencia
Águila real	Dominio aéreo	Percepción	Agilidad	Peso	Animales de Tierra en Fuerza
Oso pardo	Tanque	Fuerza	Resistencia	Velocidad	Guepardo/Tigre en Velocidad
Tiburón blanco	Depredador acuático	Ataque	Velocidad	Agilidad	Especialistas defensivos
Lobo	Versátil	Resistencia	Percepción	Peso	Especialistas de fuerza
Elefante	Coloso	Peso	Fuerza	Agilidad	Guepardo/Tigre
Rinoceronte	Muralla ofensiva	Defensa	Fuerza	Velocidad	Guepardo/Tigre
Cocodrilo	Emboscador	Ataque	Defensa	Agilidad	Velocidad/Percepción
Tigre	Duelista ofensivo	Ataque	Agilidad	Defensa	Oso/Rinoceronte
Pingüino	Resistencia acuática	Resistencia	Percepción	Fuerza/Ataque	Tiburón/Ballena
Ballena	Coloso acuático	Resistencia	Peso	Agilidad	Especialistas móviles

10. Motor de combate y valor efectivo

El motor de reglas debe ser independiente de React y Phaser. Debe recibir un estado de partida y una acción válida, devolver un nuevo estado y explicar el resultado.

Modelo conceptual

Valor efectivo = Valor base + Modificador de hábitat + Modificador de habilidad + Modificador de estado

El orden debe ser determinista. Si dos cartas empatan, el juego necesita una regla explícita: por ejemplo, empate sin daño y consumo del turno, o desempate definido por una característica secundaria. No debe quedar a criterio de la interfaz.

Datos que debe producir el motor

• Carta atacante

• Carta defensora

• Hábitat

• Atributo

• Valor base atacante

• Valor base defensor

• Modificadores aplicados

• Valor efectivo atacante

• Valor efectivo defensor

• Ganador

• Daño

• Estado aplicado

• Siguiente turno

11. Estados de carta y restricciones

Estado	Significado	Regla	UI
Disponible	Carta lista	Puede seleccionarse	Borde normal
Dañada	Ha perdido salud	Puede seguir jugando mientras no sea eliminada	Barra HP
Bloqueada	Atributo temporalmente no disponible	No puede usar ese atributo durante la ventana definida	Candado + atributo
Eliminada	Fuera de la partida	No puede seleccionarse	Carta apagada/retirada

El storyboard muestra explícitamente cartas con atributos bloqueados. Esta mecánica debe estar integrada desde el primer prototipo para no diseñar una interfaz que luego sea incompatible con las reglas.

12. Modos de juego

1 vs IA

Un jugador contra una IA que selecciona cartas defensoras respetando la información disponible.

Regla: La IA no puede conocer la carta atacante antes de escoger.

1 vs 1

Dos jugadores en una sala privada.

Regla: El servidor controla turnos y validaciones.

2 vs 2

Dos equipos de dos jugadores.

Regla: Debe existir un orden de actuación inequívoco y una asignación clara de cartas.

Condición de victoria

La condición base debe ser eliminar las cuatro cartas del equipo rival, coherente con el storyboard. En 2 vs 2, la eliminación se contabiliza sobre el conjunto del equipo.

13. Especificación de la IA

La IA no debe “adivinar” de forma mágica. Debe ejecutar una política de decisión basada en información que realmente tendría un jugador humano.

Cuando la IA es defensora

157. Recibe hábitat y atributo.

158. Consulta únicamente sus cartas disponibles.

159. Descarta cartas eliminadas.

160. Descarta cartas que tengan bloqueado el atributo seleccionado.

161. Calcula el valor efectivo de cada carta defensora posible.

162. Aplica un factor de dificultad para introducir errores o decisiones menos óptimas en niveles bajos.

163. Selecciona una carta y confirma la acción.

Niveles

Nivel	Comportamiento
Fácil	Prioriza valor medio y comete decisiones subóptimas con frecuencia.
Normal	Prioriza el mejor valor efectivo disponible con una pequeña variación.
Difícil	Considera valor efectivo, hábitat, atributos bloqueados y conservación de cartas.

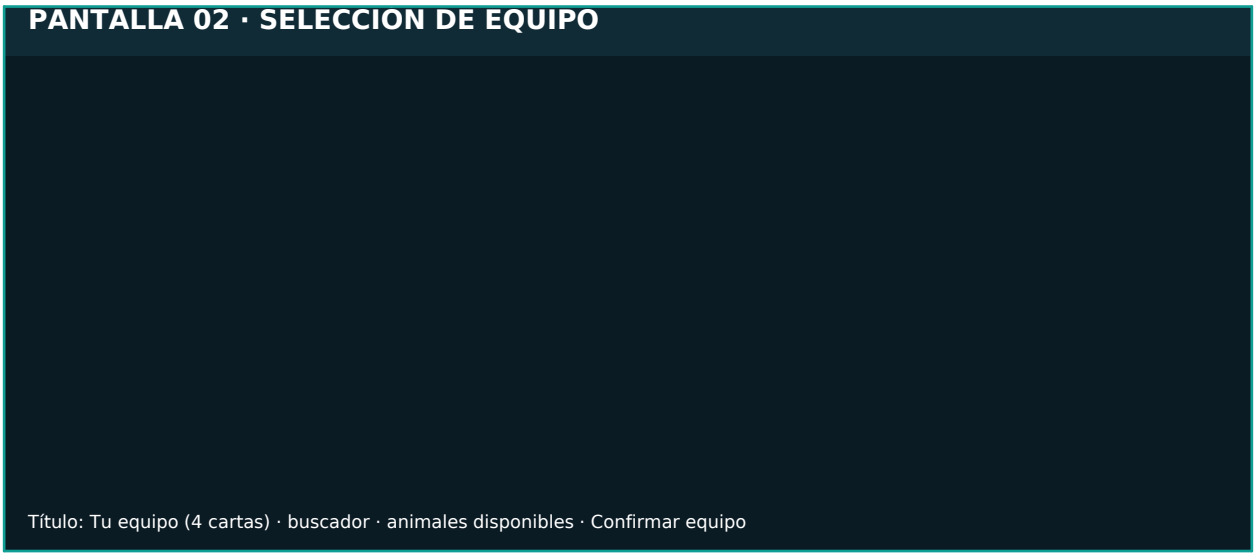
14. Mockups funcionales — mapa de pantallas

Estos mockups son especificaciones visuales de referencia. La implementación final se realiza con React y Phaser.

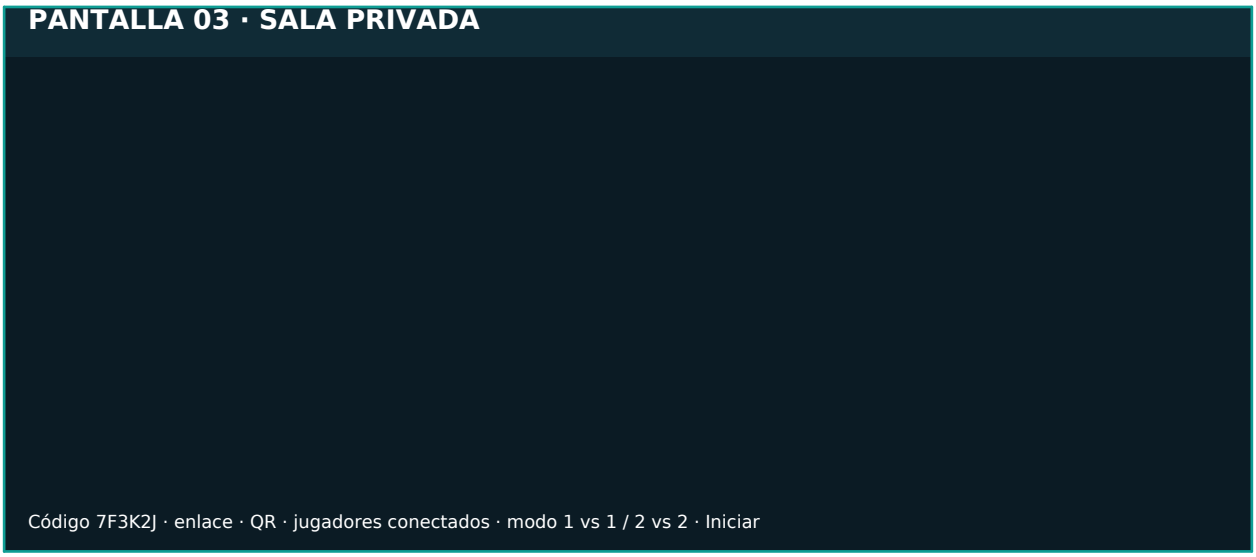
PANTALLA 01 · INICIO



PANTALLA 02 · SELECCION DE EQUIPO



PANTALLA 03 · SALA PRIVADA



15. Mockups funcionales — combate

PANTALLA 04 · ELECCION DEL ATACANTE

Paso 1: Elige hábitat [Aire] [Agua] [Tierra] [Árboles] · Paso 2: selecciona carta · Paso 3: selecciona atributo

PANTALLA 05 · RESPUESTA DEL RIVAL

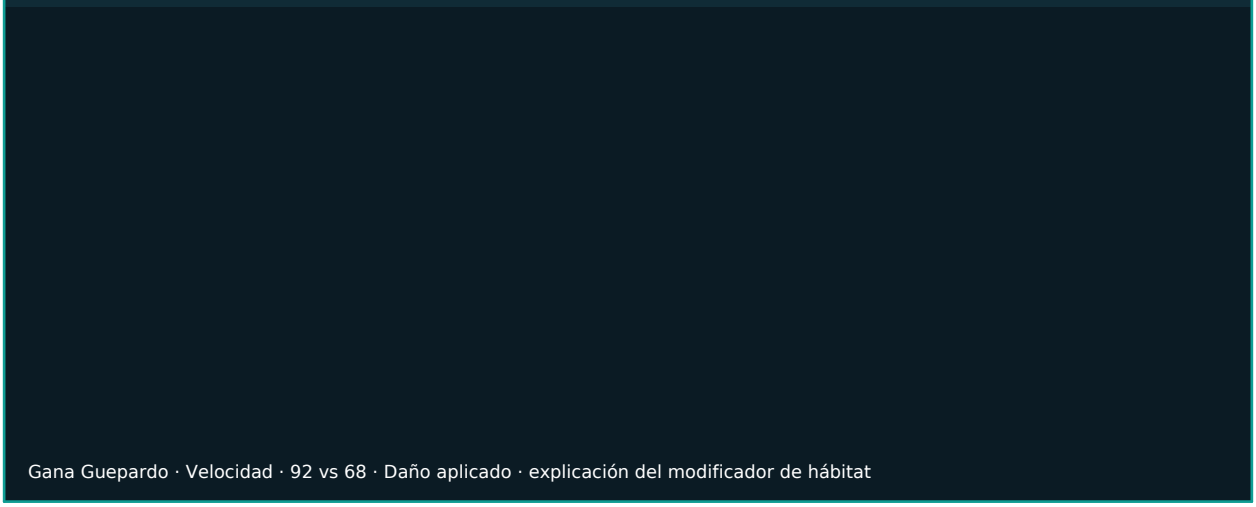
Mensaje: “Rival eligió TIERRA + VELOCIDAD” · Mostrar únicamente cartas legales para responder · Ocultar carta atacante

PANTALLA 06 · REVELACION

Revelar atacante + defensor · mostrar hábitat + atributo · valores efectivos · animación Phaser

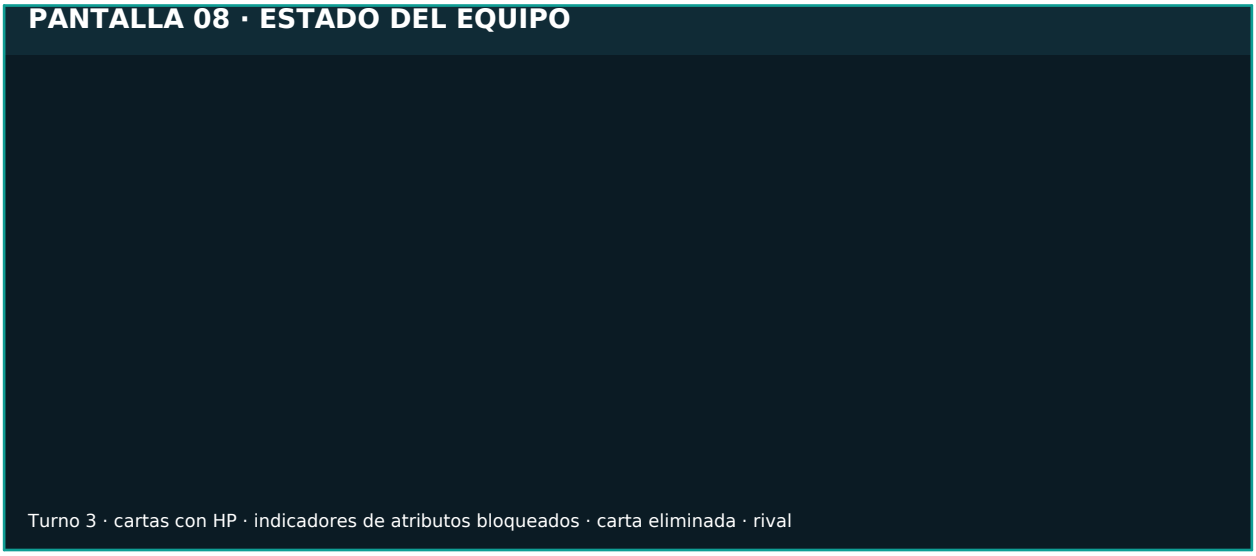
16. Mockups funcionales — resultado y estado

PANTALLA 07 · RESULTADO



Gana Guepardo · Velocidad · 92 vs 68 · Daño aplicado · explicación del modificador de hábitat

PANTALLA 08 · ESTADO DEL EQUIPO



Turno 3 · cartas con HP · indicadores de atributos bloqueados · carta eliminada · rival

PANTALLA 09 · VICTORIA



¡VICTORIA! · 4 cartas rivales eliminadas · estadísticas · volver al menú

17. Sala privada con código + QR

La sala debe poder compartirse de tres maneras: código manual, enlace y QR. El QR debe llevar al usuario directamente a la ruta de entrada a la sala.

Flujo de usuario

1. A crea la sala.
2. El servidor genera un ID interno y un código corto visible.
3. React muestra el código y construye el enlace de invitación.
4. React genera el QR a partir del enlace.
5. B abre la cámara del teléfono.
6. B escanea el QR.
7. El sistema abre ATACKNIMAL y procesa el enlace de entrada.
8. El servidor valida que la sala existe, está abierta y acepta jugadores.
9. B aparece en la sala y queda listo para jugar.

Seguridad

El QR no debe incluir contraseñas, resultados ni datos sensibles. Debe transportar un identificador o token temporal de invitación. El servidor es quien decide si la entrada es válida.

18. Sistema de imágenes reales mediante API

La primera versión utilizará fotografías reales como contenido visual de los animales. La fuente se debe registrar junto con cada recurso.

Fuentes técnicamente viables

Fuente	Uso propuesto	Consideración
Unsplash API	Buscar fotografías de animales	La documentación exige seguir sus directrices, atribución y uso de las URLs de imagen devueltas por la API.
iNaturalist API	Consultar especies/observaciones y material relacionado	API orientada a datos de biodiversidad; revisar licencias de cada contenido antes de usarlo como imagen de juego.
Wikimedia Commons/API	Buscar contenido multimedia y metadatos	Adecuado para una biblioteca de recursos con información de autor/licencia.

La documentación oficial de Unsplash describe su API JSON, búsqueda de fotos, URLs dinámicas y requisitos de atribución. cite turn0search0 turn0search2 iNaturalist documenta su API REST y recomienda la API nueva para uso escalable. cite turn0search1 turn0search4

Pipeline recomendado

- Buscar especie
- Seleccionar fotografía
- Guardar URL/fuente/licencia/autor
- Generar recorte de carta
- Cachear recurso
- Validar que la imagen siga disponible
- Mostrar atribución cuando corresponda

19. Arquitectura de software

La arquitectura se divide en cinco bloques. La separación evita que una modificación visual altere las reglas del juego.

Bloque	Tecnología	Responsabilidad
UI	React	Pantallas, navegación, formularios, selección y estado visual.
Juego visual	Phaser	Combate 2D, escenarios, sprites, animaciones, partículas.
Reglas	TypeScript independiente	Estado, acciones, cálculo y resolución.
Online	Node.js + API/WebSocket	Salas, eventos, turnos, validación y sincronización.
Persistencia	PostgreSQL	Usuarios, animales, partidas, turnos y resultados.

Regla arquitectónica

Phaser representa. El motor decide. El servidor valida. React orquesta la experiencia y presenta el estado.

20. Máquina de estados de la partida

Estado	Entrada	Acción permitida	Salida
WAITING	Sala creada	Unirse	READY
READY	Jugadores listos	Confirmar equipo	TURN_START
TURN_START	Nuevo turno	Seleccionar hábitat	ATTACK_SETUP
ATTACK_SETUP	Hábitat	Carta + atributo	DEFENSE
DEFENSE	Hábitat + atributo	Carta defensora	RESOLVE
RESOLVE	Dos cartas	Ninguna	RESULT
RESULT	Resolución	Continuar	TURN_START o FINISH
FINISH	Condición victoria	Ver resultado	MENU

21. API conceptual

No se incluyen endpoints de código; se define el contrato funcional que debe existir.

Recurso	Operación	Propósito
Sala	Crear / entrar / salir	Gestionar código, QR, participantes y estado.
Partida	Iniciar / consultar	Crear estado inicial y obtener estado actual.
Turno	Enviar acción	Validar y resolver hábitat/carta/atributo.
Animal	Listar / consultar	Obtener atributos, hábitats, habilidad e imagen.
Resultado	Consultar	Mostrar historial de una partida finalizada.

22. Modelo de datos

Entidad	Campos esenciales
Animal	id, nombre, especie, descripción, atributos, hábitats, habilidad
AnimalImage	animalId, source, url, author, license, attribution, checksum
User	id, nickname, avatar, createdAt
Team	id, userId, slot1..slot4
Room	id, code, inviteToken, mode, status, hostId, expiresAt
Match	id, roomId, mode, status, turnNumber, winnerTeam
Turn	matchId, turn, attacker, habitat, attribute, attackCard, defenseCard
CombatResult	turnId, baseValues, modifiers, effectiveValues, winner, damage, states

23. Plan de pruebas

La calidad debe medirse en tres niveles: reglas, experiencia y red.

Prueba	Objetivo	Resultado esperado
Reglas unitarias	Validar fórmulas	Mismo input → mismo resultado
Información oculta	Validar defensa	Carta atacante invisible hasta confirmar defensa
Bloqueo	Atributos restringidos	No puede usarse atributo bloqueado
IA	Decisión legal	IA nunca hace trampas
Sala QR	Entrada móvil	Escaneo abre la sala correcta
Reconexión	Pérdida de red	Estado se recupera sin duplicar acciones
Balance	Miles de simulaciones	Detectar dominancia por animal/atributo/hábitat

24. Balance mediante simulación

Antes de probar el juego con muchas personas, el motor debe permitir simular miles de enfrentamientos.

Métricas obligatorias

• Win rate por animal

• Win rate por atributo

• Win rate por hábitat

• Win rate del primer jugador

• Daño medio

• Duración media

• Frecuencia de uso de cada atributo

• Frecuencia de eliminación

• Tasa de empate

• Rendimiento de la IA

Umbral de alerta inicial

Como regla de QA, cualquier animal que supere de forma sostenida el rango objetivo definido por el equipo de diseño debe entrar a revisión. No se debe fijar un 50% exacto como dogma: el modo, la composición de equipos y el primer turno pueden introducir asimetrías.

25. UX/UI y accesibilidad

• Texto legible en móvil.

• No usar color como único indicador de estado.

• Icono + texto para atributos.

• Contraste suficiente en cartas y botones.

• Animaciones con opción de reducir movimiento.

• Botones de decisión claramente separados.

• Confirmación antes de ejecutar una elección irreversible.

• Estado de turno siempre visible.

• Hábitat y atributo elegidos deben quedar destacados.

• El resultado debe explicar la comparación.

26. Plan de desarrollo paso a paso

Paso	Módulo	Entrega
1	**Motor de reglas**	Implementar estado, turnos, atributos, hábitat, cálculo, daño, bloqueo y victoria.
2	**Roster**	Crear los 11 animales y sus ocho atributos.
3	**Balance**	Simular enfrentamientos y ajustar estadísticas.
4	**React base**	Inicio, selección de equipo, perfil y navegación.
5	**Phaser**	Escenario de combate, revelación y animaciones.
6	**Modo IA**	Integrar política de defensa y dificultad.
7	**Salas**	Código, enlace y QR.
8	**1 vs 1**	Sincronización y servidor autoritativo.
9	**2 vs 2**	Turnos de equipos y condiciones de victoria.
10	**QA**	Pruebas funcionales, móviles, red y balance.

27. Pipeline de contenido de animales

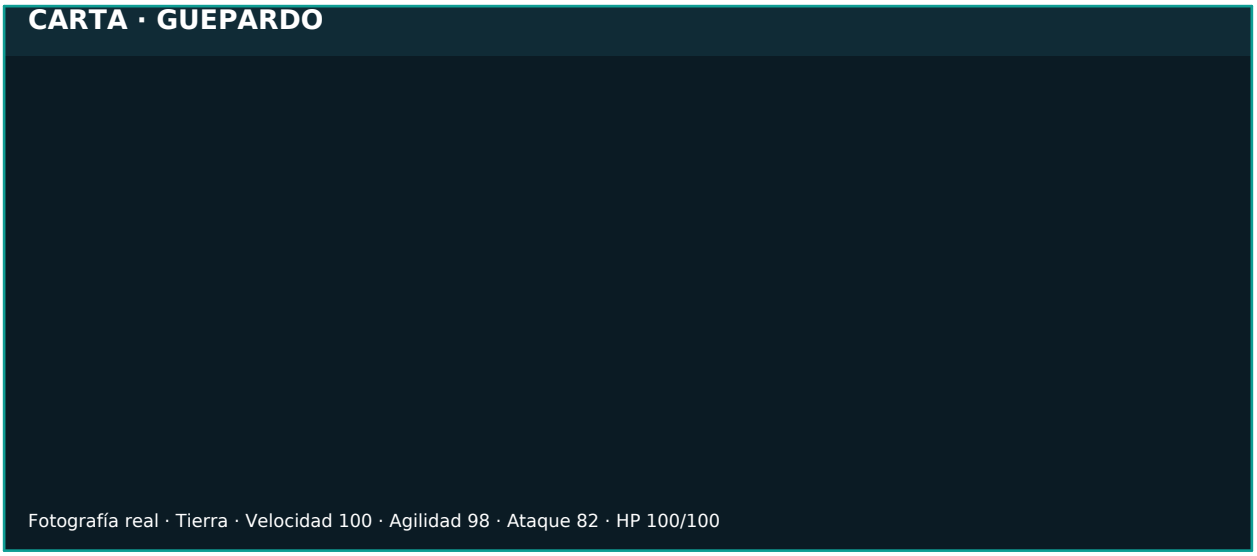
Cada animal debe pasar por una ficha maestra antes de aparecer en una partida.

Campo	Ejemplo	Responsable
Nombre	Guepardo	Diseño
Especie	Acinonyx jubatus	Contenido
Hábitats	Tierra	Diseño
8 atributos	1–100	Balance
Imagen	URL + ID + fuente	Contenido
Licencia	Licencia + atribución	Legal/Contenido
Habilidad	Texto y efecto	Diseño/Balance
Estado	Draft / QA / Published	Producción

28. Mockup de carta — especificación

Una carta debe permitir comprender al animal en menos de tres segundos.

Zona	Contenido	Regla visual
Cabecera	Nombre + especie	Nombre grande, alto contraste
Imagen	Fotografía real	Recorte consistente 4:3
Hábitat	Iconos de afinidad	Máximo cuatro indicadores
Atributos	8 valores	Icono + nombre + número
Salud	HP actual / máximo	Barra y número
Habilidad	Nombre + descripción	Mostrar solo cuando exista



29. Reglas específicas del 2 vs 2

El modo 2 vs 2 no debe improvisarse sobre el 1 vs 1. Debe definir un orden de actuación explícito.

Propuesta inicial

Turno	Acción
1	Jugador A1 ataca; B1 defiende
2	Jugador B2 ataca; A2 defiende
3	Jugador A2 ataca; B2 defiende
4	Jugador B1 ataca; A1 defiende

Este orden es una propuesta de diseño para el primer prototipo. Debe validarse mediante pruebas porque el equilibrio del 2 vs 2 dependerá de qué cartas controle cada integrante y de cómo se distribuyan los turnos.

30. Errores y estados de red

- Jugador pierde conexión durante su ventana de elección.
- Jugador intenta enviar dos acciones.
- Jugador intenta enviar una carta eliminada.
- Jugador intenta usar atributo bloqueado.
- Sala expirada.
- Código inválido.
- QR con token expirado.
- Servidor recibe acción fuera de turno.

En todos los casos, el cliente debe recibir un estado comprensible y el servidor debe conservar la autoridad sobre la partida.

31. Fuentes de imágenes y control de licencias

La aplicación debe conservar metadatos de cada imagen. No basta con guardar una URL.

Campo	Descripción
sourceProvider	Proveedor
sourceId	ID del recurso
author	Autor de la fotografía
license	Licencia aplicable
attribution	Texto requerido de atribución
sourceUrl	Página original
imageUrl	URL utilizada por la aplicación

Nunca publicar una imagen simplemente porque una API la devuelve. La disponibilidad técnica no equivale automáticamente a permiso de reutilización para un videojuego.

32. Referencias técnicas

Documentación oficial consultada para la arquitectura de imágenes reales:

• Unsplash API Documentation — documentación oficial de la API, búsqueda, URLs de imagen, límites y requisitos de atribución. □url□Unsplash API Documentation□turn0search0□

• Unsplash API Guidelines — directrices oficiales actualizadas en 2026 sobre hotlinking, uso de URLs y descarga. □url□Unsplash API Guidelines□turn0search2□

• iNaturalist API — documentación oficial de endpoints REST para observaciones y datos de biodiversidad. □url□iNaturalist API□turn0search1□

• iNaturalist API Recommended Practices — recomendaciones oficiales para utilizar la API nueva. □url□API Recommended Practices□turn0search4□

33. Checklist de producto

1. Los 11 animales están definidos.
2. Los 8 atributos están definidos.
3. Cada animal tiene fortalezas y debilidades.
4. Los 4 hábitats están definidos.
5. El atacante elige hábitat + carta + atributo.
6. El defensor solo conoce hábitat + atributo antes de elegir.
7. La carta atacante permanece oculta.
8. El motor calcula el valor efectivo.
9. El daño y bloqueo quedan registrados.
10. Se puede eliminar una carta.
11. La victoria elimina las 4 cartas rivales.
12. Existe modo 1 vs IA.
13. Existe modo 1 vs 1.
14. Existe modo 2 vs 2.
15. Existe sala por código.
16. Existe enlace.
17. Existe QR.
18. El QR puede abrir la sala desde cámara.
19. Phaser solo presenta el combate.
20. React controla la aplicación.
21. El servidor valida partidas online.
22. PostgreSQL conserva el estado persistente.
23. Las imágenes tienen fuente/licencia.
24. No existe economía en esta versión.
25. El balance se prueba con simulaciones.

34. Conclusión de diseño

ATACKNIMAL debe construirse alrededor de una idea sencilla pero profunda: el atacante declara el contexto y el atributo; el defensor debe responder sin conocer la carta atacante. Esa regla crea el espacio estratégico del juego.

La diversidad no viene de tener cientos de animales desde el comienzo. Viene de que un roster pequeño esté cuidadosamente balanceado y produzca decisiones distintas. Los 11 animales iniciales permiten crear especialistas de velocidad, percepción, fuerza, defensa, resistencia, ataque y peso, distribuidos entre distintos hábitats.

La arquitectura React + Phaser permite separar claramente la experiencia de usuario del combate visual. El motor de reglas es el corazón lógico; el servidor se convierte en autoridad cuando existe juego online; y la base de datos conserva el estado. El sistema de imágenes reales mediante API mantiene una identidad visual basada en animales auténticos, siempre respetando la licencia y atribución de cada recurso.

Objetivo de la primera versión: que una persona pueda entrar, formar un equipo de cuatro animales, iniciar una partida, tomar decisiones estratégicas, defenderse sin conocer la carta rival, ver un combate espectacular y entender perfectamente por qué ganó o perdió.

ATACKNIMAL queda así definido como un juego de estrategia por turnos, de información parcial, con animales reales, tres modos de juego y una arquitectura web moderna preparada para crecer sin sacrificar la claridad de las reglas.